

Gestione delle Immagini e Politiche di Pull

Obiettivi di Apprendimento

Al termine di questa sezione il corsista sarà in grado di:

- Configurare correttamente `imagePullPolicy` nei Pod (`Always`, `IfNotPresent`, `Never`)
- Gestire i pull secret a livello di Pod e di ServiceAccount
- Aggiornare il pull secret globale del cluster per aggiungere credenziali per registry privati
- Configurare `allowedRegistries` e `blockedRegistries` nel cluster per limitare l'uso di registry non autorizzati
- Creare e applicare un `ImageDigestMirrorSet` (IDMS) per reindirizzare trasparentemente il pull verso un mirror ACR
- Verificare l'autenticità delle immagini con `cosign` e configurare una `ClusterImagePolicy`
- Integrare ACR con Managed Identity per il pull senza credenziali statiche su ARO

Teoria e Concetti Chiave

imagePullPolicy: Quando e Come OpenShift Tira l'Immagine

Il campo `spec.containers[].imagePullPolicy` nel manifest del Pod controlla quando kubelet tenta di scaricare l'immagine:

Valore	Comportamento	Quando usarlo
<code>Always</code>	Tenta sempre di scaricare l'immagine prima di avviare il container	Tag mutabili come <code>:latest</code> ; ambienti dove è critico avere sempre l'ultima versione
<code>IfNotPresent</code>	Usa la cache del nodo se l'immagine è già presente; altrimenti scarica	Tag immutabili (es. <code>1.2.3</code> o digest sha256); migliore performance
<code>Never</code>	Non scarica mai l'immagine; fallisce se non è già presente nel nodo	Ambienti offline, immagini pre-caricate

⚠ Attenzione: Se non si specifica `imagePullPolicy` e il tag è `:latest`, OpenShift imposta automaticamente `Always`. Per tutti gli altri tag, imposta `IfNotPresent`. Per evitare comportamenti inaspettati, **specificare sempre esplicitamente** il campo.

💡 Suggerimento: In produzione, evitare il tag `:latest` per le immagini applicative. Usare tag semantici (`1.2.3`) o digest sha256 (`image@sha256:...`) con `imagePullPolicy: IfNotPresent`.

Pull Secret: Autenticazione ai Registry

I **pull secret** contengono le credenziali (username/password o token) per autenticarsi ai registry privati.

Livelli di configurazione dei pull secret:

Livello	Come configurare	Scope
Pod	<code>spec.imagePullSecrets[].name</code>	Solo quel Pod
ServiceAccount	<code>imagePullSecrets[].name</code> su SA	Tutti i Pod che usano quel SA
Cluster globale	<code>pull-secret</code> in <code>openshift-config</code>	Tutti i nodi del cluster (system-level)

Il pull secret è un Secret di tipo `kubernetes.io/dockerconfigjson` con la struttura:

```
{
  "auths": {
    "registry.example.com": {
      "auth": "<base64(username:password)>",
      "username": "myuser",
      "password": "mypassword",
      "email": "optional@example.com"
    }
  }
}
```

Registri Consentiti e Bloccati

La risorsa `image.config.openshift.io/cluster` (singleton del cluster) permette di definire una lista allowlist/blocklist dei registry:

```
spec:
  allowedRegistries:                                # ← se impostato, SOLO questi registry
sono consentiti
  - domainName: registry.redhat.io
    insecure: false
  - domainName: myaroregistry.azurecr.io
    insecure: false
  blockedRegistries:                                # ← questi registry sono sempre
bloccati
  - domainName: docker.io
    insecure: false
```

⚠ Attenzione: Impostare `allowedRegistries` è un'operazione **critica di cluster**. Se si dimentica di includere i registry interni OpenShift (es. `quay.io`, `registry.redhat.io`), i Pod di sistema non riusciranno a fare pull e il cluster potrebbe diventare instabile.

ImageDigestMirrorSet (IDMS) e ImageContentSourcePolicy (ICSP)

L'**ImageDigestMirrorSet** (introdotto in OCP 4.13, sostituisce l'ICSP deprecato) permette di **redirigere trasparentemente** il pull di immagini da un registry pubblico verso un mirror privato (es. ACR), senza modificare i manifest delle applicazioni.

Flusso di risoluzione con IDMS:

1. Il Pod specifica **registry.access.redhat.com/ubi9/ubi-minimal:9.4**
2. kubelet consulta la configurazione mirror locale (iniettata dall'Operator MachineConfig nei nodi)
3. Tenta prima il pull da **myaroregistry.azurecr.io/redhat/ubi9/ubi-minimal:9.4** (mirror)
4. Se fallisce (cache miss), tenta il registry originale come fallback

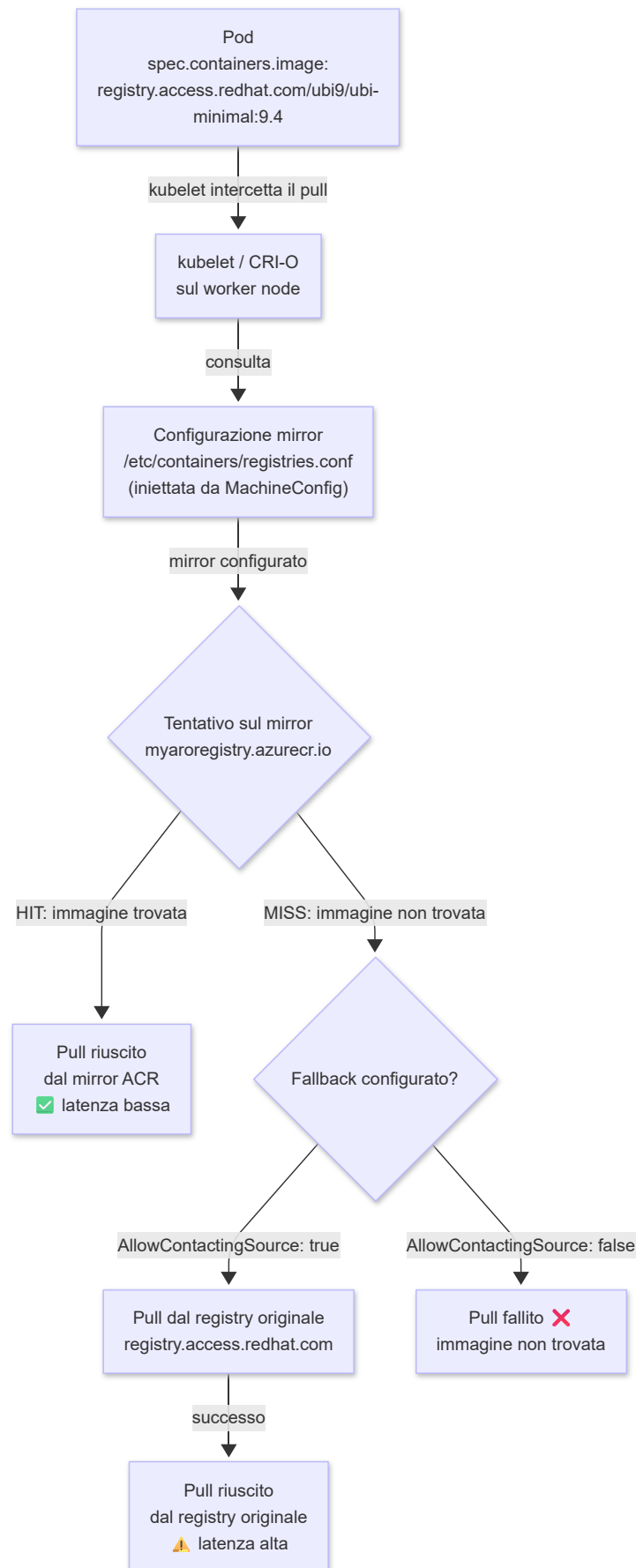
Firma delle Immagini con cosign e ClusterImagePolicy

A partire da OCP 4.13, OpenShift supporta la verifica della firma delle immagini tramite **cosign** (progetto Sigstore). La **ClusterImagePolicy** definisce le regole di verifica applicate a livello di cluster.

Oggetto	Scope	Versione OCP
ClusterImagePolicy	Tutto il cluster	OCP 4.13+
ImagePolicy	Singolo namespace	OCP 4.13+

Diagrammi

Risoluzione dell'Immagine con ImageDigestMirrorSet



Comandi Pratici con Output Atteso

```
# Verificare la configurazione attuale dei registry consentiti/bloccati nel
cluster
oc get image.config.openshift.io/cluster -o yaml | \
  grep -A 10 "registrySources"

# Output atteso (default: nessuna restrizione):
# registrySources: {}
```

```
# Bloccare il pull da docker.io nel cluster
oc patch image.config.openshift.io/cluster \
  --type=merge \
  --patch='{ "spec":{ "registrySources":{ "blockedRegistries":["docker.io"]} } }'

# Output atteso:
# image.config.openshift.io/cluster patched
```

```
# Verificare il pull secret globale del cluster (quali registry sono autenticati)
oc get secret pull-secret -n openshift-config \
  -o jsonpath='{.data.\.dockerconfigjson}' | \
  base64 -d | jq '.auths | keys'

# Output atteso:
# [
#   "arosvc.azurecr.io",
#   "registry.connect.redhat.com",
#   "registry.redhat.io"
# ]
```

```
# Aggiungere credenziali ACR al pull secret globale del cluster
# ATTENZIONE: operazione critica – non sovrascrivere il secret esistente
EXISTING=$(oc get secret pull-secret -n openshift-config \
  -o jsonpath='{.data.\.dockerconfigjson}' | base64 -d)

NEW_AUTH=$(echo -n "myuser:mypassword" | base64 -w0)

# Aggiungere le nuove credenziali al JSON esistente
echo "${EXISTING}" | jq --arg auth "${NEW_AUTH}" \
  '.auths["myaroregistry.azurecr.io"] = {"auth": $auth}' > /tmp/new-pull-
secret.json

# Aggiornare il secret globale
oc set data secret/pull-secret \
  -n openshift-config \
```

```
--from-file=.dockerconfigjson=/tmp/new-pull-secret.json
```

```
# Output atteso:  
# secret/pull-secret data updated
```

```
# Verificare gli ImageDigestMirrorSet configurati nel cluster  
oc get imagedigestmirrorset
```

```
# Output atteso:  
# NAME          AGE  
# redhat-registry 5d
```

```
# Ispezionare un'immagine su ACR con skopeo (verifica layers, digest,  
configurazione)  
skopeo inspect \  
  docker://myaroregistry.azurecr.io/ubi9/ubi-minimal:9.4 \  
  --creds $(az acr credential show --name myaroregistry --query username -o  
tsv):$(az acr credential show --name myaroregistry --query "passwords[0].value" -o  
tsv)
```

```
# Output atteso:  
# {  
#   "Name": "myaroregistry.azurecr.io/ubi9/ubi-minimal",  
#   "Digest": "sha256:a1b2c3...",  
#   "RepoTags": ["9.4", "latest"],  
#   "Architecture": "amd64",  
#   "Os": "linux",  
#   ...  
# }
```

```
# Importare un'immagine pubblica in ACR (operazione di mirroring server-side)  
az acr import \  
  --name myaroregistry \  
  --source registry.access.redhat.com/ubi9/ubi-minimal:9.4 \  
  --image ubi9/ubi-minimal:9.4
```

```
# Output atteso:  
# Importing from: registry.access.redhat.com/ubi9/ubi-minimal:9.4 ...  
# Done
```

```
# Verificare la firma di un'immagine con cosign  
cosign verify \  
  --certificate-identity-regexp=".*" \  
  --certificate-oidc-issuer-regexp=".*" \  
  registry.access.redhat.com/ubi9/ubi-minimal:9.4
```

```
# Output atteso:  
# Verification for registry.access.redhat.com/ubi9/ubi-minimal:9.4 --  
# The following checks were performed on each of these signatures:  
#   - The cosign claims were validated  
#   - The signatures were verified against the specified public key
```

Esempi YAML Completi

ImageDigestMirrorSet + ClusterImagePolicy per Verifica Firma

```
# ImageDigestMirrorSet: reindirige il pull di immagini Red Hat verso un mirror ACR
# NOTA: IDMS sostituisce ImageContentSourcePolicy (ICSP) deprecato da OCP 4.13
# IMPORTANTE: la modifica viene applicata tramite MachineConfig a tutti i nodi
(richiede reboot)
apiVersion: config.openshift.io/v1
kind: ImageDigestMirrorSet
metadata:
  name: redhat-registry-mirror
spec:
  imageDigestMirrors:
    # Mirror per registry.access.redhat.com → ACR
    - source: registry.access.redhat.com      # ← registry originale da reindirigere
      mirrors:
        - myaroregistry.azurecr.io/redhat    # ← mirror ACR (directory prefisso)
      mirrorSourcePolicy: AllowContactingSource # ← AllowContactingSource =
fallback al sorgente                                     # ← NeverContactSource = solo dal
mirror (offline)

    # Mirror per registry.redhat.io → ACR (componenti Red Hat)
    - source: registry.redhat.io
      mirrors:
        - myaroregistry.azurecr.io/redhat-io
      mirrorSourcePolicy: AllowContactingSource

    # Mirror per quay.io → ACR (immagini community)
    - source: quay.io
      mirrors:
        - myaroregistry.azurecr.io/quay
      mirrorSourcePolicy: AllowContactingSource
---
# ClusterImagePolicy: verifica la firma cosign delle immagini di produzione
# Richiede OCP 4.13+ con la funzionalità SigstoreImageVerification abilitata
apiVersion: config.openshift.io/v1alpha1
kind: ClusterImagePolicy
metadata:
  name: verify-production-images
spec:
  scopes:
    # Applica la verifica alle immagini del namespace "production"
    - myaroregistry.azurecr.io/production/* # ← pattern del registry/repository

  policy:
    rootOfTrust:
      # Tipo di root of trust: PublicKey | FulcioCAWithRekor | PKIFailOpen
      policyType: PublicKey # ← verifica con chiave pubblica cosign
```



```
publicKey:
  # Chiave pubblica cosign (PEM) usata per firmare le immagini
  keyData: |                                     # ← chiave pubblica PEM in base64
    -----BEGIN PUBLIC KEY-----
    MFkwEwYHKOZIZj0CAQYIKoZIZj0DAQcDQgAE...
    -----END PUBLIC KEY-----
  hashAlgorithm: sha256                         # ← algoritmo hash della firma
```

Note Specifiche per Azure ARO

Managed Identity per Pull da ACR Senza Secret Statici

La soluzione più sicura su ARO è usare la **Managed Identity** del cluster per autenticarsi ad ACR, eliminando la necessità di credenziali statiche nel pull secret:

```
# Recuperare il Service Principal del cluster ARO
ARO_SP=$(az aro show \
  --name myaro \
  --resource-group myaro-rg \
  --query servicePrincipalProfile.clientId -o tsv)

# Recuperare l'ID della risorsa ACR
ACR_ID=$(az acr show \
  --name myaroregistry \
  --resource-group myaro-rg \
  --query id -o tsv)

# Assegnare il ruolo AcrPull al Service Principal del cluster ARO sull'ACR
az role assignment create \
  --assignee ${ARO_SP} \
  --scope ${ACR_ID} \
  --role AcrPull

# Output atteso:
# {
#   "id": "/subscriptions/.../roleAssignments/...",
#   "principalId": "...",
#   "roleDefinitionId": "...",
#   ...
# }
```

Dopo aver assegnato il ruolo, i nodi ARO possono fare pull da ACR senza credenziali statiche nel pull secret.

az acr import: Mirror Server-Side delle Immagini

az acr import copia un'immagine da un registry pubblico (es. registry.access.redhat.com) direttamente nell'ACR, senza passare per la rete locale:

```
# Importare le builder image S2I in ACR per ambienti ARO disconnessi
az acr import \
  --name myaroregistry \
  --source registry.access.redhat.com/ubi9/openjdk-21:1.19 \
  --image ubi9/openjdk-21:1.19

az acr import \
  --name myaroregistry \
  --source registry.access.redhat.com/ubi9/nodejs-20:1 \
  --image ubi9/nodejs-20:1

az acr import \
  --name myaroregistry \
  --source registry.access.redhat.com/ubi9/ubi-minimal:9.4 \
  --image ubi9/ubi-minimal:9.4
```

Riepilogo dei Punti Chiave

- **imagePullPolicy**: **Always** forza il pull ad ogni riavvio del container (default per **:latest**); **IfNotPresent** usa la cache del nodo (consigliato per tag semantici); **Never** richiede che l'immagine sia pre-caricata
 - I pull secret possono essere configurati a livello di Pod, ServiceAccount o cluster globale (**openshift-config/pull-secret**)
 - **image.config.openshift.io/cluster** controlla **allowedRegistries** e **blockedRegistries** a livello di cluster; una configurazione errata può rendere instabile il cluster
 - **ImageDigestMirrorSet** (IDMS, OCP 4.13+) redirige trasparentemente il pull verso un mirror (es. ACR) senza modificare i manifest; sostituisce l'ICSP deprecato
 - **ClusterImagePolicy** (OCP 4.13+) abilita la verifica delle firme cosign/sigstore per garantire l'autenticità delle immagini
 - Su ARO, il pull da ACR con **Managed Identity** (ruolo **AcrPull** sul Service Principal) è più sicuro dei secret statici
 - **az acr import** copia immagini server-side da registry pubblici nell'ACR: ideale per ambienti disconnessi o con egress limitato
-

Domande di Verifica

1. Qual è il comportamento di default di `imagePullPolicy` quando il tag dell'immagine è `:latest`?

- A) `IfNotPresent`
- B) `Never`
- C) `Always` ✓
- D) `OnStartup`

2. Quale oggetto OpenShift (OCP 4.13+) permette di redirigere trasparentemente il pull di immagini da un registry pubblico verso un mirror ACR senza modificare i Deployment?

- A) `ImageContentSourcePolicy` (deprecato)
- B) `ImageDigestMirrorSet` ✓
- C) `ImageStreamImportPolicy`
- D) `RegistryMirrorConfig`

3. Come si aggiunge in modo sicuro le credenziali ACR al pull secret globale del cluster senza sovrascrivere le credenziali esistenti?

- A) `oc delete secret pull-secret -n openshift-config && oc create secret ...`
- B) Si legge il secret esistente, si aggiungono le nuove credenziali al JSON, e si usa `oc set data secret/pull-secret` con il JSON aggiornato ✓
- C) `oc patch secret pull-secret --type=merge --patch='{ "data": { "acr": "..."} }'`
- D) Non è possibile aggiungere credenziali aggiuntive al pull secret globale

4. Quale ruolo Azure deve essere assegnato al Service Principal del cluster ARO per permettere il pull da ACR senza credenziali statiche?

- A) `Contributor`
- B) `AcrPush`
- C) `AcrPull` ✓
- D) `Reader`

5. Cosa accade ai nodi del cluster quando si applica un nuovo `ImageDigestMirrorSet`?

- A) Nessun impatto: la configurazione è applicata immediatamente senza riavvii
- B) Solo il Pod del registry viene riavviato
- C) L'Operator MachineConfig aggiorna la configurazione CRI-O sui nodi; i nodi vengono riavviati in modo rolling ✓
- D) È necessario ricreare manualmente tutti i Pod del cluster